



Business Intelligence Analytics

The end of the year 2015, a Pizza business is looking to analyse the performance of the business over the year. They are interested in tracking KPI performance, trends over time and visualization that shows these trends.



My Process

Overview

This project involved developing an interactive Power BI dashboard to analyze sales performance and key business KPIs. By consolidating revenue, profit, orders, customer, product, and regional data into a single reporting solution, the dashboard enables stakeholders to monitor performance, identify trends, and make informed, data-driven decisions.

Business Problem Statement(EDA)

KPI REQUIREMENT

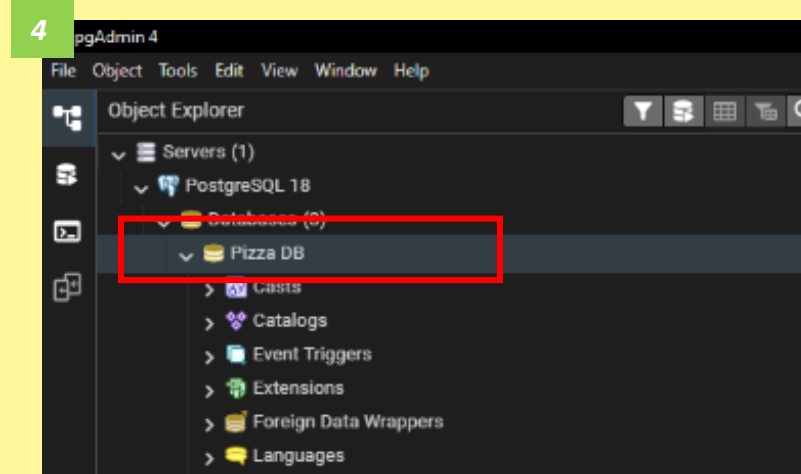
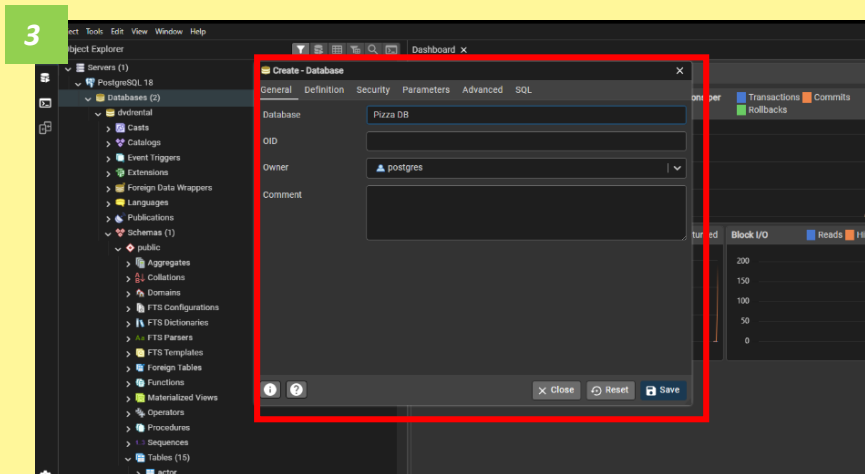
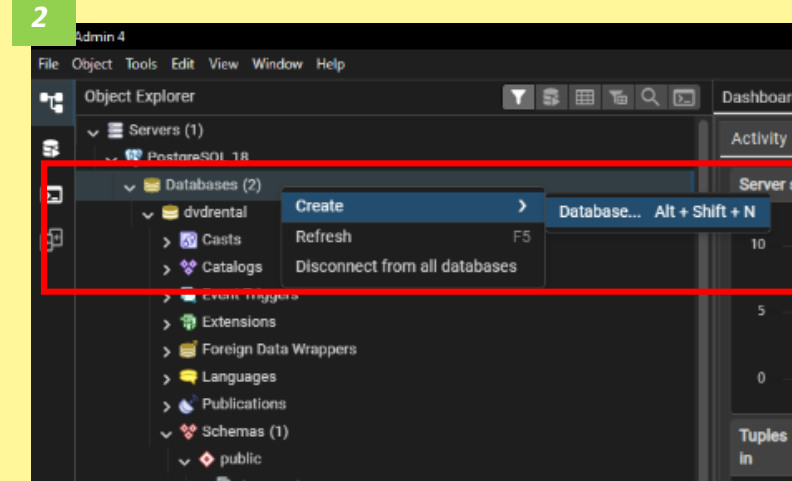
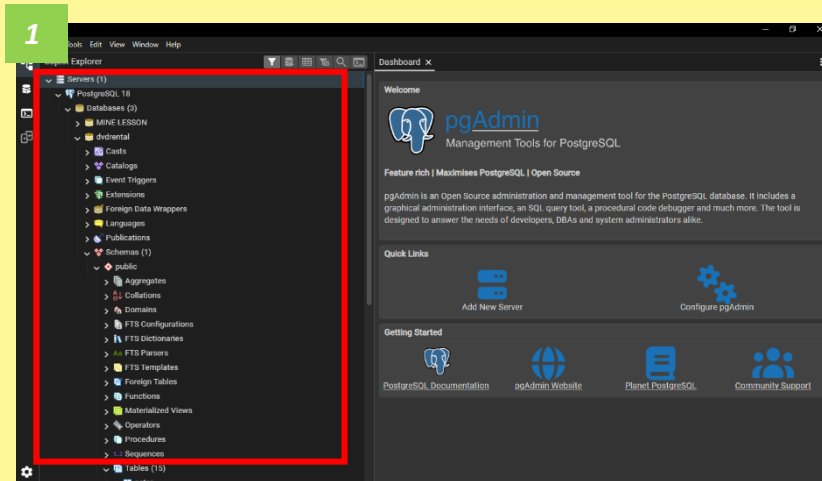
1. **TOTAL REVENUE:** The sum of total price of all pizza orders
2. **AVERAGE ORDER VALUE:** The average amount spent per order, calculated by dividing the total revenue by the total number of orders
3. **TOTAL PIZZAS SOLD:** The sum of the quantities of all pizza sold
4. **TOTAL ORDERS:** The total number of orders placed.
5. **AVG PIZZAS PER ORDER:** The avg number of pizzas sold per order, calculated by dividing the total number of pizzas sold by the total number of orders

Business Problem Statement(EDA)

CHARTS REQUIREMENT

6. Daily Trend for Total Orders
7. Monthly Trend for total Orders
8. Percentage of Sales by Pizza Category
9. Percentage of Sales by Pizza size
10. Top 5 Best sellers by Revenue, Total Quantity and total orders
11. Bottom 5 worst sellers by Revenue, Total Quantity and total orders

1. Creating DataBase



2. Creating Table and importing csv file

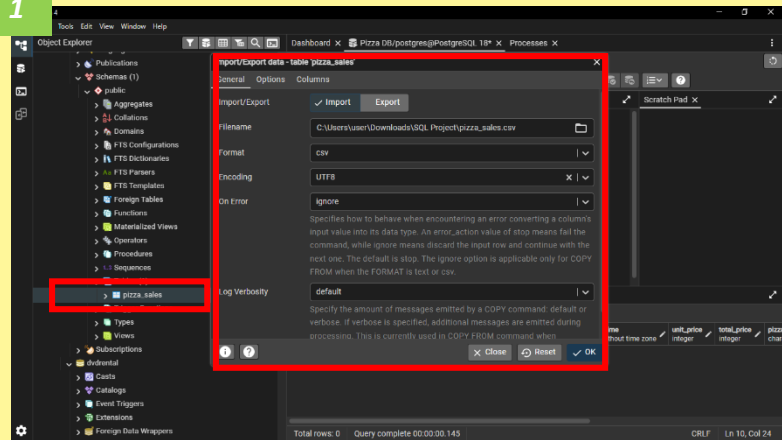
```
1  Create table pizza_sales(  
2  pizza_id int primary key,  
3  order_id int,  
4  pizza_name_id varchar (50),  
5  quantity int,  
6  order_date date,  
7  order_time time,  
8  unit_price int,  
9  total_price int,  
0  pizza_size varchar (5),  
1  pizza_category varchar (50),  
2  pizza_ingredients varchar (500),  
3  pizza_name varchar (50)  
4  );  
5  select * from
```

SQL to create a table, note
all columns must match
with that of the intended
data to import

3. Importing File, Data Cleaning and resolving errors with importing csv file

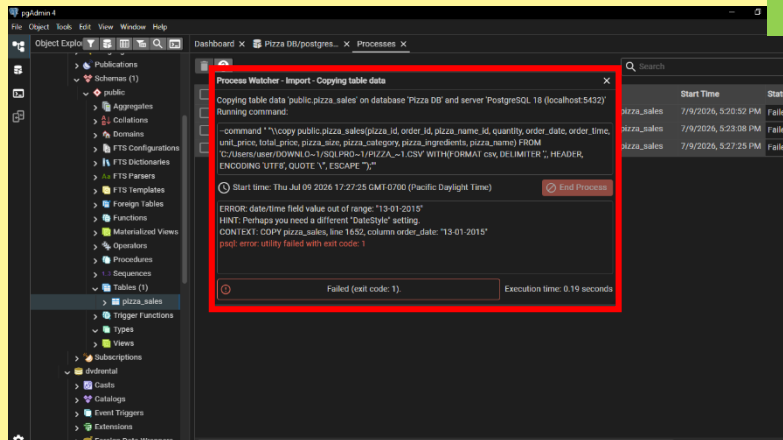
1

1st attempt
to import
csv file



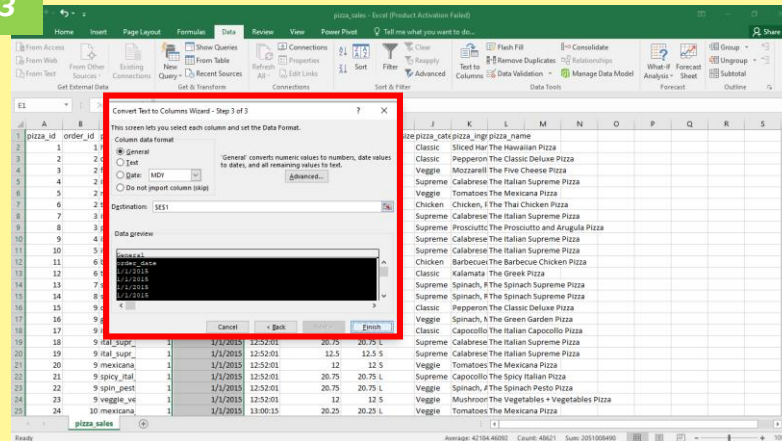
2

Failed to
import.
Error message



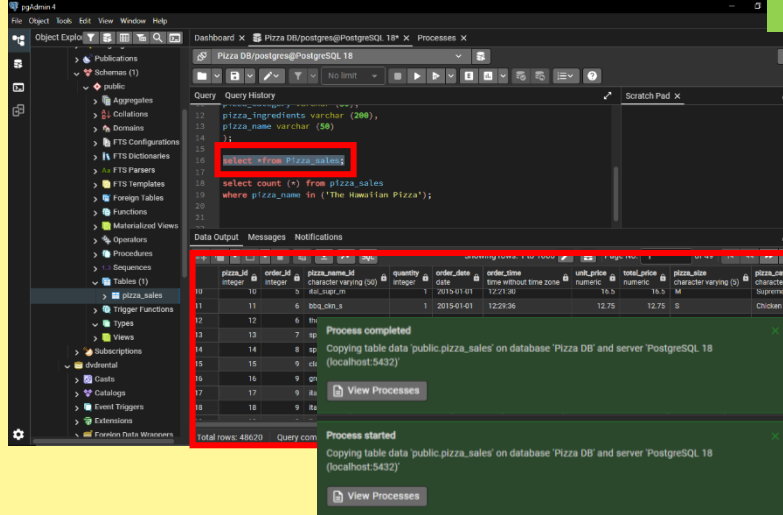
3

Cleaned file
with Excel



4

2nd attempt was
successful with
a confirmation
pop up



pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurations
 - FTS Dictionaries
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Views
 - Operators
 - Procedures
 - Sequences
 - Tables (1)
 - pizza_sales
 - Trigger Functions
 - Types
 - Views
- Subscriptions
- dvdrental
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers

Dashboard x Pizza DB/postgres@PostgreSQL 18* x Processes x

Pizza DB/postgres@PostgreSQL 18

Query Query History

```
12 pizza_ingredients varchar (200),
13 pizza_name varchar (50)
14 );
15
16 select * from Pizza_sales;
17
18 select count (*) from pizza_sales
19 where pizza_name in ('The Hawaiian Pizza');
```

Scratch Pad x

Data Output Messages Notifications

Showing rows: 1 to 1000 Page No: 1 of 49

	pizza_id integer	order_id integer	pizza_name_id character varying (50)	quantity integer	order_date date	order_time time without time zone	unit_price numeric	total_price numeric	pizza_size character varying (5)	pizza_cate character v
10	10	5	ital_supr_m	1	2015-01-01	12:21:30	16.5	16.5	M	Supreme
11	11	6	bbq_ckn_s	1	2015-01-01	12:29:36	12.75	12.75	S	Chicken
12	12	6	the_greek_s	1	2015-01-01	12:29:36	12	12	S	Classic
13	13	7	spinach_supr_s	1	2015-01-01	12:50:37	12.5	12.5	S	Supreme
14	14	8	spinach_supr_s	1	2015-01-01	12:51:37	12.5	12.5	S	Supreme
15	15	9	classic_dlx_s	1	2015-01-01	12:52:01	12	12	S	Classic
16	16	9	green_garden_s	1	2015-01-01	12:52:01	12	12	S	Veggie
17	17	9	ital_cpello_l	1	2015-01-01	12:52:01	20.5	20.5	L	Classic
18	18	9	ital_supr_l	1	2015-01-01	12:52:01	20.75	20.75	L	Supreme
--	--	-	--	-	-----	-----	---	---	-	-

Total rows: 48620 Query complete 00:00:00.351 CRLF Ln 16, Col 1

ANSWERING BUSINESS PROBLEM STATEMENT

After cleaning and importing the data, we move to the
querying proper

1. TOTAL REVENUE

```
20
21 SELECT SUM(total_price) As Total_Revenue from pizza_sales;
22
23
24
```

Data Output		Messages	Notifications
total_revenue numeric			
1	817860.05		

2. AVERAGE ORDER VALUE

```
SELECT SUM(total_price) / COUNT(DISTINCT order_id) as Avg_Order_Value from pizza_sales;
```

Data Output		Messages	Notifications
avg_order_value numeric			
	38.3072622950819672		

3. TOTAL PIZZAS SOLD

```
24
25 SELECT SUM(quantity) as Total_Pizza_Sold from pizza_sales;
```

Data Output		Messages	Notifications
total_pizza_sold bigint			
	49574		

4. TOTAL ORDERS

```
26
27 SELECT COUNT(DISTINCT order_id) as Total_Order from pizza_sales;
28
```

Data Output		Messages	Notifications
total_order bigint			
1	21350		

5. AVG PIZZAS PER ORDER

```
SELECT CAST (CAST (SUM(quantity) AS DECIMAL (10,2)) /  
CAST (COUNT(DISTINCT order_id) AS DECIMAL (10,2)) AS DECIMAL (10,2))  
AS Avg_Pizza_Per_Order FROM pizza_sales;
```

Data Output Messages Notifications

+ [File Icon] [Dropdown] [Clipboard Icon] [Dropdown] [Trash Icon] [Database Icon] [Download Icon] [Line Graph Icon] SQL Showing rows: 1 to 1 [Edit Icon]

avg_pizza_per_order
numeric (10,2) 🔒

2.32

6. Daily Trend for Total Orders

```
40 SELECT
41     TO_CHAR(order_date, 'Dy') AS order_day,
42     COUNT(DISTINCT order_id) AS total_orders
43 FROM pizza_sales
44 GROUP BY TO_CHAR(order_date, 'Dy')
45 ORDER BY MIN(EXTRACT(DOW FROM order_date));
```

Data Output Messages Notifications



	order_day text	total_orders bigint
1	Sun	2624
2	Mon	2794
3	Tue	2973
4	Wed	3024
5	Thu	3239
6	Fri	3538
7	Sat	3158

7. Monthly Trend for total Orders

```
46 SELECT
47     TO_CHAR(order_date, 'Mon') AS order_month,
48     COUNT(DISTINCT order_id) AS total_orders
49 FROM pizza_sales
50 GROUP BY TO_CHAR(order_date, 'Mon'), EXTRACT(MONTH FROM order_date)
51 ORDER BY EXTRACT(MONTH FROM order_date);
```

Data Output Messages Notifications



Showing rows: 1 to 1

	order_month text	totalOrders bigint
1	Jan	1845
2	Feb	1685
3	Mar	1840
4	Apr	1799
5	May	1853
6	Jun	1773
7	Jul	1935
8	Aug	1841
9	Sep	1661
10	Oct	1646
11	Nov	1792
12	Dec	1680

Total rows: 12 Query complete 00:00:00.497

8. Percentage of Sales by Pizza Category

```
56
57 SELECT pizza_category, CAST(SUM(total_price) * 100.0 /
58 (SELECT SUM(total_price) FROM pizza_sales)
59 AS DECIMAL(10,2)) AS percentage_of_sales
60 FROM pizza_sales
61 GROUP BY pizza_category;
```

	pizza_category character varying (50)	percentage_of_sales numeric (10,2)
1	Supreme	25.46
2	Chicken	23.96
3	Veggie	23.68
4	Classic	26.91

9. Percentage of Sales by Pizza size

```
63
64 SELECT pizza_size, CAST(SUM(total_price) * 100.0 /
65 (SELECT SUM(total_price) FROM pizza_sales)
66 AS DECIMAL(10,2)) AS percentage_of_sales
67 FROM pizza_sales
68 GROUP BY pizza_size;
```

	pizza_size character varying (5)	percentage_of_sales numeric (10,2)
1	S	21.77
2	XXL	0.12
3	XL	1.72
4	M	30.49
5	L	45.89

10. Total Pizzas sold by pizza category

```
6  select pizza_category, count(distinct order_id)
7  as Total_orders from pizza_sales
8  group by pizza_category
9  order by Total_orders desc
10 limit 5
```

Data Output Messages Notifications



pizza_category character varying (50) 🔒	total_orders bigint 🔒
Classic	10859
Supreme	9085
Veggie	8941
Chicken	8536

11. Top 5 Best sellers by Revenue, Total Quantity and total orders

```
80
81 SELECT pizza_name, SUM(Quantity) Total_Quantity from pizza_sales
82 group by pizza_name
83 Order by Total_Quantity DESC
84 LIMIT 5
```

	pizza_name character varying (50)	total_quantity bigint
1	The Classic Deluxe Pizza	2453
2	The Barbecue Chicken Piz...	2432
3	The Hawaiian Pizza	2422
4	The Pepperoni Pizza	2418
5	The Thai Chicken Pizza	2371

Total rows: 5 Query complete 00:00:00.139

```
71 SELECT pizza_name, SUM(total_price) Total_Revenue from pizza_sales
72 group by pizza_name
73 Order by Total_Revenue DESC
74 LIMIT 5
```

	pizza_name character varying (50)	total_revenue numeric
1	The Thai Chicken Pizza	43434.25
2	The Barbecue Chicken Piz...	42768.00
3	The California Chicken Piz...	41409.50
4	The Classic Deluxe Pizza	38180.5
5	The Spicy Italian Pizza	34831.25

Showing rows: 1 to 5

```
90
91 SELECT pizza_name, COUNT(DISTINCT order_id) AS Total_orders from pizza_sales
92 group by pizza_name
93 Order by Total_orders DESC
94 LIMIT 5
```

	pizza_name character varying (50)	total_orders bigint
1	The Classic Deluxe Pizza	2329
2	The Hawaiian Pizza	2280
3	The Pepperoni Pizza	2278
4	The Barbecue Chicken Piz...	2273
5	The Thai Chicken Pizza	2225

Showing rows: 1 to 5

12. Bottom 5 worst sellers by Revenue, Total Quantity and total orders

```
76 SELECT pizza_name, SUM(total_price) Total_Revenue from pizza_sales
77 group by pizza_name
78 Order by Total_Revenue ASC
79 LIMIT 5
```

	pizza_name character varying (50)	total_revenue numeric
1	The Brie Carre Pizza	11588.50
2	The Green Garden Pizza	13955.75
3	The Spinach Supreme Piz...	15277.75
4	The Mediterranean Pizza	15360.50
5	The Spinach Pesto Pizza	15596.00

```
81 SELECT pizza_name, SUM(Quantity) Total_Quantity from pizza_sales
82 group by pizza_name
83 Order by Total_Quantity ASC
84 LIMIT 5
```

	pizza_name character varying (50)	total_quantity bigint
1	The Brie Carre Pizza	490
2	The Mediterranean Pizza	934
3	The Calabrese Pizza	937
4	The Spinach Supreme Piz...	950
5	The Soppressata Pizza	961

Total rows: 5 Query complete 00:00:00.126

```
86 SELECT pizza_name, COUNT(DISTINCT order_id) AS Total_orders from pizza_sales
87 group by pizza_name
88 Order by Total_orders
89 LIMIT 5
```

	pizza_name character varying (50)	total_orders bigint
1	The Brie Carre Pizza	480
2	The Mediterranean Pizza	912
3	The Calabrese Pizza	918
4	The Spinach Supreme Piz...	918
5	The Chicken Pesto Pizza	938

Conclusion

- In the course of this project I faced a number of blockers, most of which were at the initial stage and affected the total number of rows imported into PostgreSQL and also in some cases stopped the importation with an error message.
- With my good research skills and with Excel I was able to locate and clean the data allowing me to move forward.
- Queries like WHERE, GROUP BY, ORDER BY, CASE were particularly handy in this project.
- Although the JOIN statement wasn't used here it remains another very powerful tool in SQL.

Thank you for your time.
I would be more than happy to discuss
further and address any concerns you
may have.

adiidanielu@gmail.com 0806 795 4519